Setor de Educação Profissional e Tecnológica
Tecnologia em Análise e Desenvolvimento de Sistemas

DS143 - Estruturas de Dados II

12 - Representação e percursos em grafos

Prof. Alex Kutzke

Outubro de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Usar a terminologia básica de grafos: vértice, aresta, grau, caminho, ciclo, grafo conexo, componente conexa, grafo dirigido;
2. Representar um grafo por matriz e por listas de adjacência, e escolher entre as duas;
3. Executar à mão a busca em largura e a busca em profundidade, registrando a ordem de visita e o vetor `pai`;
4. Explicar por que a busca em largura encontra o caminho com menos arestas, e a busca em profundidade não;
5. Usar a busca em profundidade para encontrar componentes conexas e detectar ciclos;
6. Obter uma ordenação topológica pela pós-ordem invertida;
7. Reconhecer as componentes fortemente conexas de um grafo dirigido pequeno.

Este conteúdo cai na Prova 2.

Roteiro

Grafos

Representação

Buscas

Aplicações

Grafos dirigidos

Fechamento

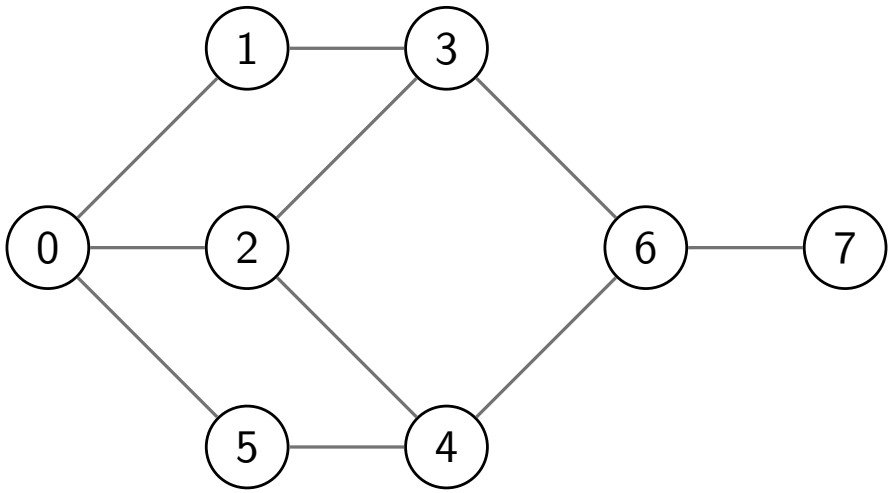
Demonstração: um labirinto

```
$ go run labirinto.go
Busca em largura                    Busca em profundidade
  ######################              ######################
  #S....   #          #              #S.......#          #
  #    .   #   ####   #              #        .#   ####   #
  #####.   #      #   #              #####    .#      #   #
  #  ...   #####  #   #              #  ......#####  #   #
  #  .###             #              #  .###............#
  #E..  #             #              #E..  #............#
  ######################              ######################
  13 passos, 43 células visitadas     45 passos, 48 células visitadas
```

- a mesma regra nas duas: de uma célula livre se passa para as células livres vizinhas;
- a busca em largura achou o caminho mais curto; a em profundidade, um caminho qualquer.

Grafo

Um **grafo** $G = (V, E)$: um conjunto V de **vértices** e um conjunto E de **arestas**, cada uma ligando dois vértices.



- 8 vértices, 10 arestas. V e E também indicam as quantidades: "a busca custa $O(V + E)$";
- o desenho não faz parte do grafo: o que define o grafo é a lista de arestas.

Terminologia

Termo	Definição	No grafo
adjacentes (vizinhos)	ligados por uma aresta	2 e 4
grau	número de arestas que tocam o vértice	3 tem grau 3
caminho	sequência de vértices, cada um adjacente ao seguinte	0, 2, 4, 6
comprimento	número de arestas do caminho	3
ciclo	caminho que volta ao início, sem repetir arestas	0, 1, 3, 2, 0
conexo	existe caminho entre quaisquer dois vértices	sim
componente conexa	conjunto máximo de vértices ligados por caminhos	uma só

Sem laço (aresta de v para v) e sem arestas paralelas.

Graus, densidade e árvores

- cada aresta soma 1 ao grau de cada ponta: a soma dos graus é $2E$. No exemplo, $3+2+3+3+3+2+3+1=20$;
- no máximo $V(V-1)/2$ arestas. Perto disso, grafo **denso**; com E da ordem de V, **esparso**;
- um cruzamento de ruas tem 4 vizinhos, e não milhares: a maior parte dos grafos de aplicações é esparsa.

- **árvore**: grafo conexo e sem ciclos, com $V-1$ arestas. As árvores das aulas anteriores, com uma raiz escolhida;
- em um grafo qualquer pode haver mais de um caminho entre dois vértices. É isso que muda os algoritmos;
- **grafo dirigido**: o arco $v \to w$ vai de v a w, e não volta.

Onde aparecem

Aplicação	Vértices	Arestas
malha viária	cruzamentos	ruas; dirigido com mão única
rede de computadores	roteadores	enlaces
circuito eletrônico	portas lógicas	fios
rede social	pessoas	amizade; dirigido se "seguir"
jogo de tabuleiro	posições	jogadas permitidas
biologia	proteínas	interação entre proteínas
web	páginas	links, dirigido
artigos científicos	artigos	"A cita B", dirigido
dependências	pacotes	"A importa B", dirigido
memória de um programa	objetos	ponteiros, dirigido
labirinto, grade	células livres	passagem entre vizinhas

Exemplos dos slides de Sedgewick e Wayne e desta aula. As duas próximas aulas acrescentam um **peso** a cada aresta. Nesta, as arestas não têm peso.

Matriz de adjacência

```
type Matriz struct {
  adj [][]bool
}

func (g *Matriz) AdicionaAresta(v, w int) {
  g.adj[v][w] = true
  g.adj[w][v] = true
}
```

```
    0 1 2 3 4 5 6 7
 0  . 1 1 . . 1 . .
 1  1 . . 1 . . . .
 2  1 . . 1 1 . . .
 3  . 1 1 . . . 1 .
 4  . . 1 . . 1 1 .
 5  1 . . . 1 . . .
 6  . . . 1 1 . . 1
 7  . . . . . . 1 .
```

- vértices numerados de 0 a $V - 1$, como os itens do Union-Find;
- cada aresta aparece duas vezes: a matriz é simétrica;
- a aresta v-w existe? Uma consulta, `adj[v][w]`;
- V^2 posições, quaisquer que sejam as arestas.

Listas de adjacência

```
type Grafo struct {
  adj [][]int
}

func NovoGrafo(n int) *Grafo {
  return &Grafo{adj: make([][]int, n)}
}

func (g *Grafo) V() int { return len(g.adj) }

func (g *Grafo) AdicionaAresta(v, w int) {
  g.adj[v] = append(g.adj[v], w)
  g.adj[w] = append(g.adj[w], v)
}
```

```
0: [1 2 5]
1: [0 3]
2: [0 3 4]
3: [1 2 6]
4: [2 5 6]
5: [0 4]
6: [3 4 7]
7: [6]
```

- total de posições nas listas: a soma dos graus, $2E$;
- as buscas visitam os vizinhos na ordem da lista.

Matriz ou listas

Operação	Matriz	Listas
memória	V^2	$V + 2E$
inserir aresta	$O(1)$	$O(1)$
a aresta v-w existe?	$O(1)$	$O(\text{grau}(v))$
percorrer os vizinhos de v	$O(V)$	$O(\text{grau}(v))$

1 milhão de cruzamentos, 4 vizinhos em média:

- listas: 4×10^6 inteiros de 8 bytes (32 MB), mais 24 bytes de cabeçalho por *slice* (24 MB);
- matriz de `bool`: 10^{12} bytes, 1 TB.

Buscas em grafo esparso: listas. Grafo denso, ou muita pergunta "a aresta existe?": matriz.

Lista de arestas e vértices com nome

- a **lista de arestas**, como a entrada dos programas, é uma terceira representação. Não serve para buscas; é a entrada do algoritmo de Kruskal, na próxima aula;
- vértices com nome: um `map` de nome para número, e um *slice* para o caminho inverso.

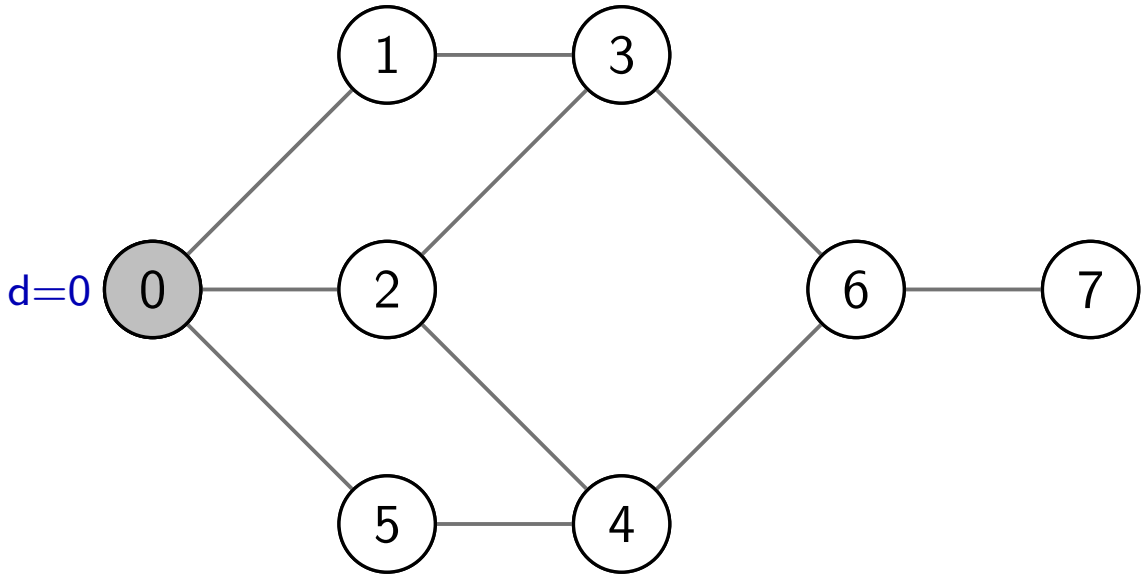
```
var indice = map[string]int{}
var nomes []string

func numero(nome string) int {
  if v, ok := indice[nome]; ok {
    return v
  }
  indice[nome] = len(nomes)
  nomes = append(nomes, nome)
  return len(nomes) - 1
}
```

Busca em largura (BFS)

```
func BuscaEmLargura(g *Grafo, s int) *BFS {
  b := &BFS{marcado: make([]bool, g.V()),
    pai: make([]int, g.V()), dist: make([]int, g.V())}
  b.pai[s] = -1
  b.marcado[s] = true
  fila := []int{s}
  for len(fila) > 0 {
    v := fila[0]
    fila = fila[1:]
    for _, w := range g.adj[v] {
      if !b.marcado[w] {
        b.marcado[w] = true
        b.pai[w] = v
        b.dist[w] = b.dist[v] + 1
        fila = append(fila, w)
      }
    }
  }
  return b
}
```

Busca em largura passo a passo



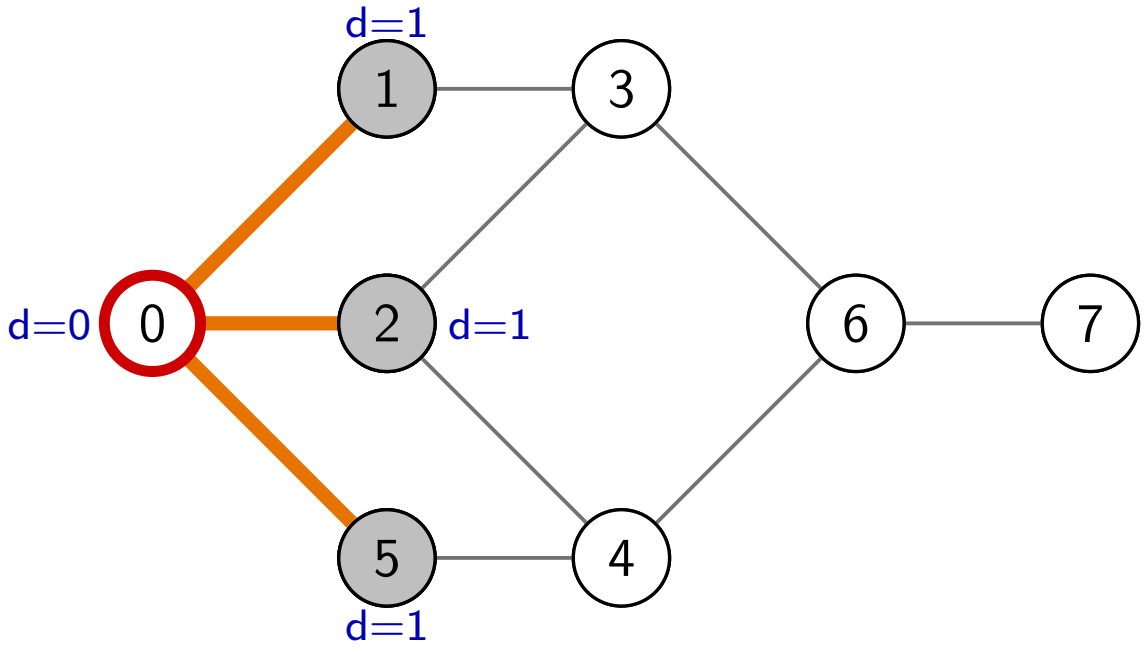
v	dist	pai
0	0	-1
1		
2		
3		
4		
5		
6		
7		

fila: 0

início: marca o 0 e o põe na fila

na fila já retirado retirado agora

Busca em largura passo a passo



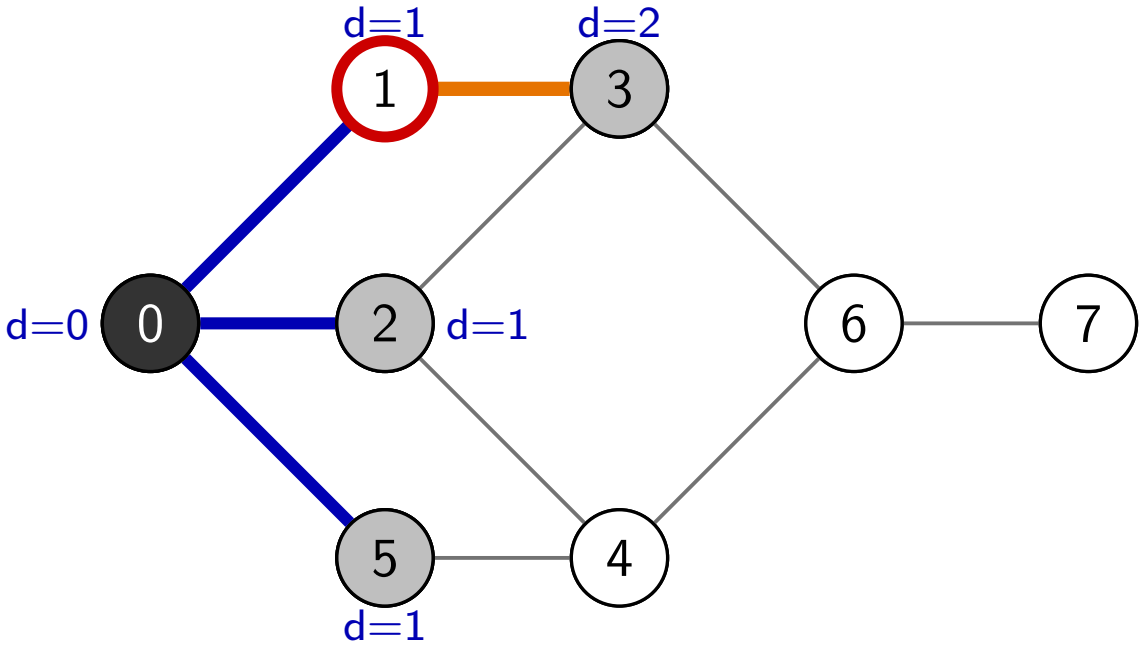
v	dist	pai
0	0	-1
1	1	0
2	1	0
3		
4		
5	1	0
6		
7		

fila: 1 2 5

retira 0, lista [1 2 5]: 1, 2 e 5 entram na fila

na fila já retirado retirado agora

Busca em largura passo a passo



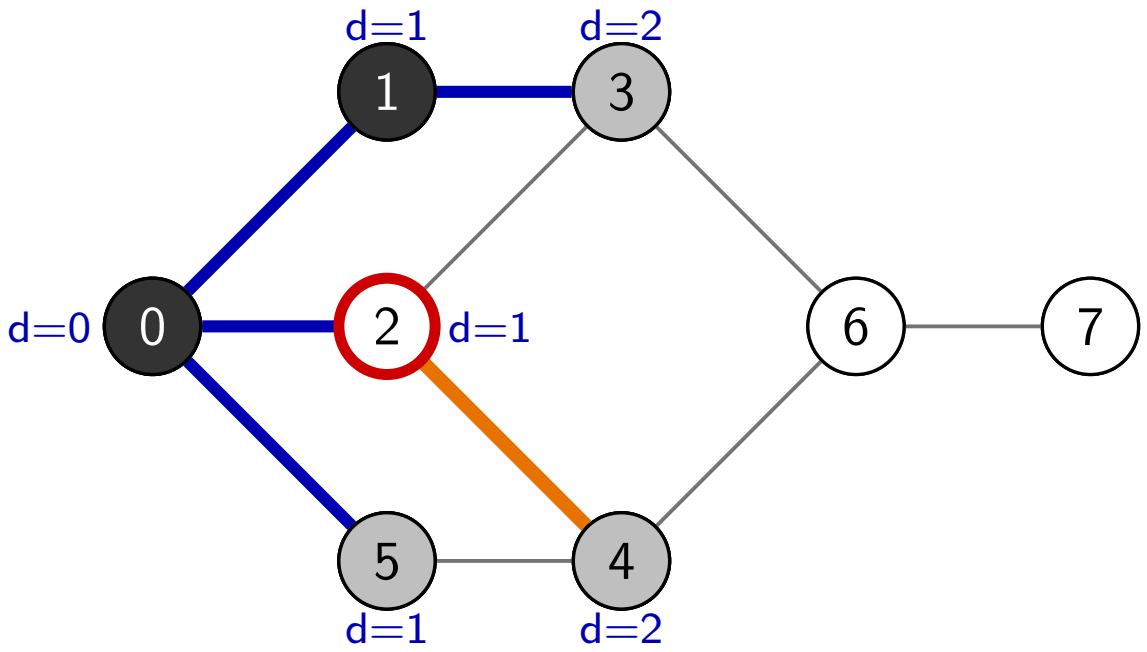
v	dist	pai
0	0	-1
1	1	0
2	1	0
3	2	1
4		
5	1	0
6		
7		

fila: 2 5 3

retira 1, lista [0 3]: 3 entra na fila

na fila já retirado retirado agora

Busca em largura passo a passo



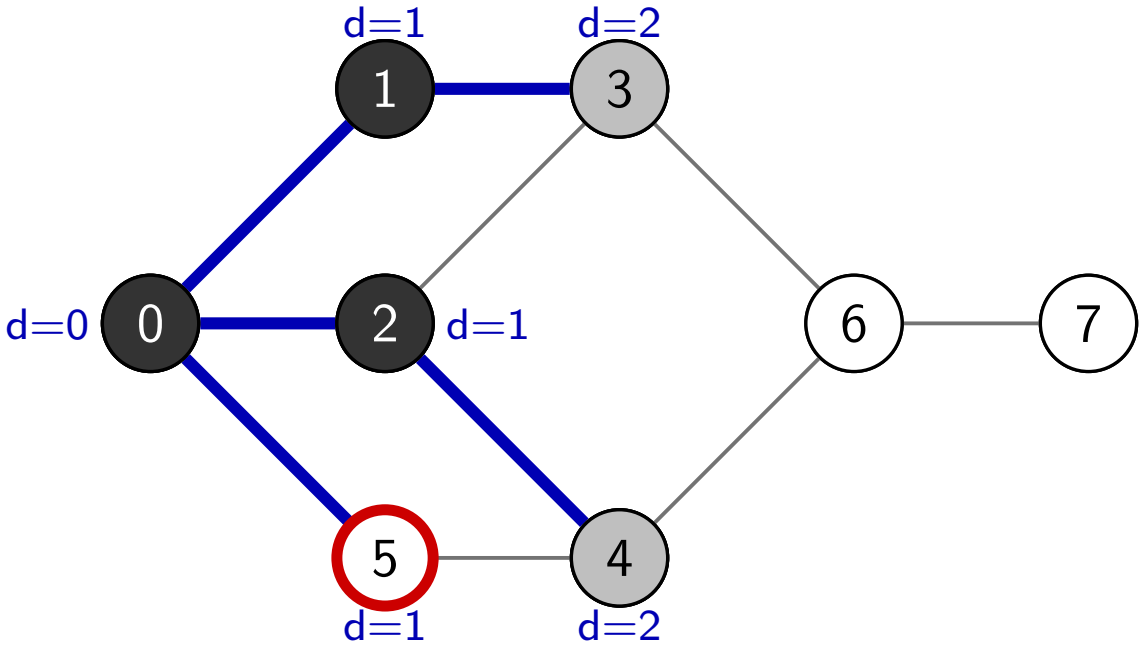
v	dist	pai
0	0	-1
1	1	0
2	1	0
3	2	1
4	2	2
5	1	0
6		
7		

fila: 5 3 4

retira 2, lista [0 3 4]: 4 entra na fila

na fila já retirado retirado agora

Busca em largura passo a passo



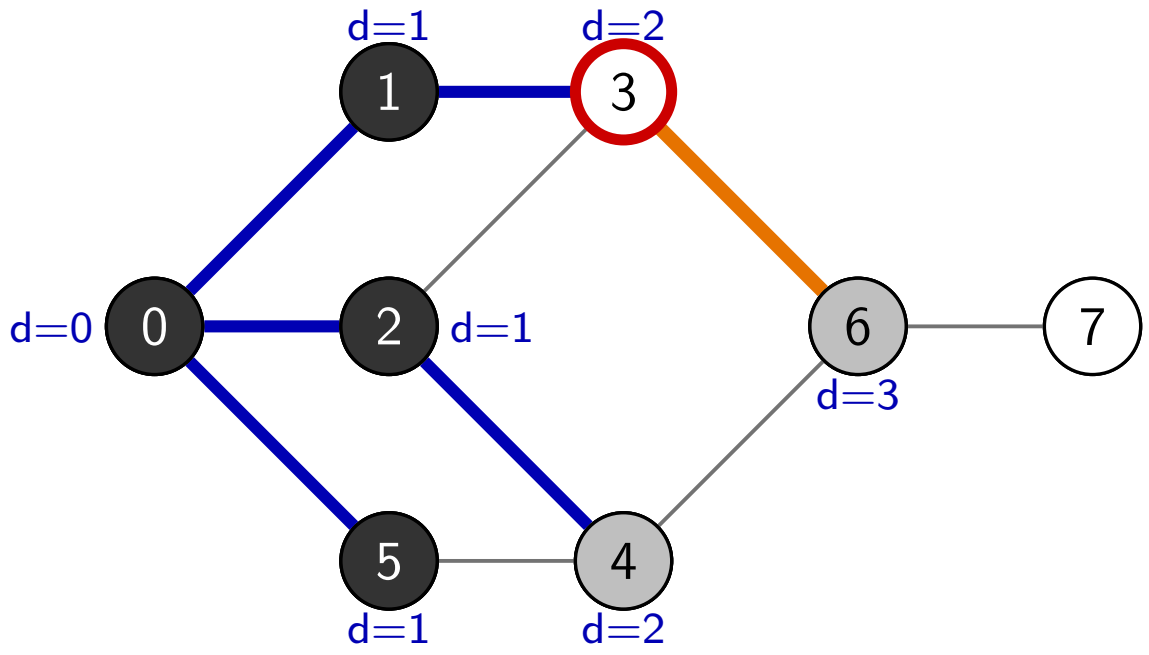
v	dist	pai
0	0	-1
1	1	0
2	1	0
3	2	1
4	2	2
5	1	0
6		
7		

fila: 3 | 4

retira 5, lista [0 4]: todos marcados, nada entra

na fila já retirado retirado agora

Busca em largura passo a passo



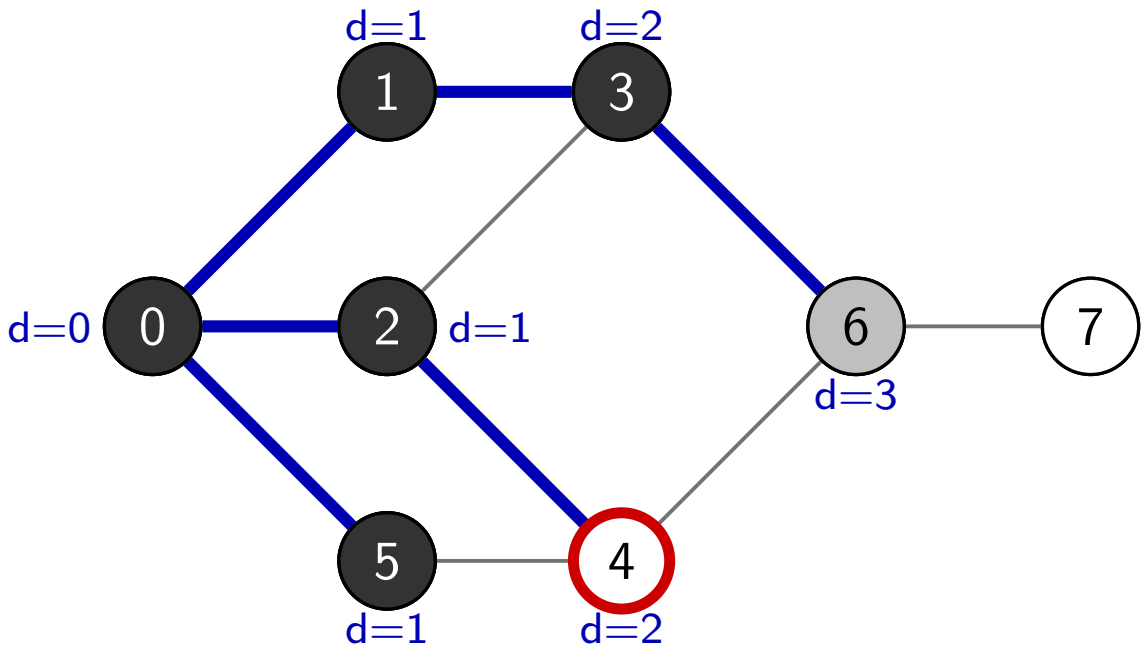
v	dist	pai
0	0	-1
1	1	0
2	1	0
3	2	1
4	2	2
5	1	0
6	3	3
7		

fila: 4 6

retira 3, lista [1 2 6]: 6 entra na fila

na fila já retirado retirado agora

Busca em largura passo a passo



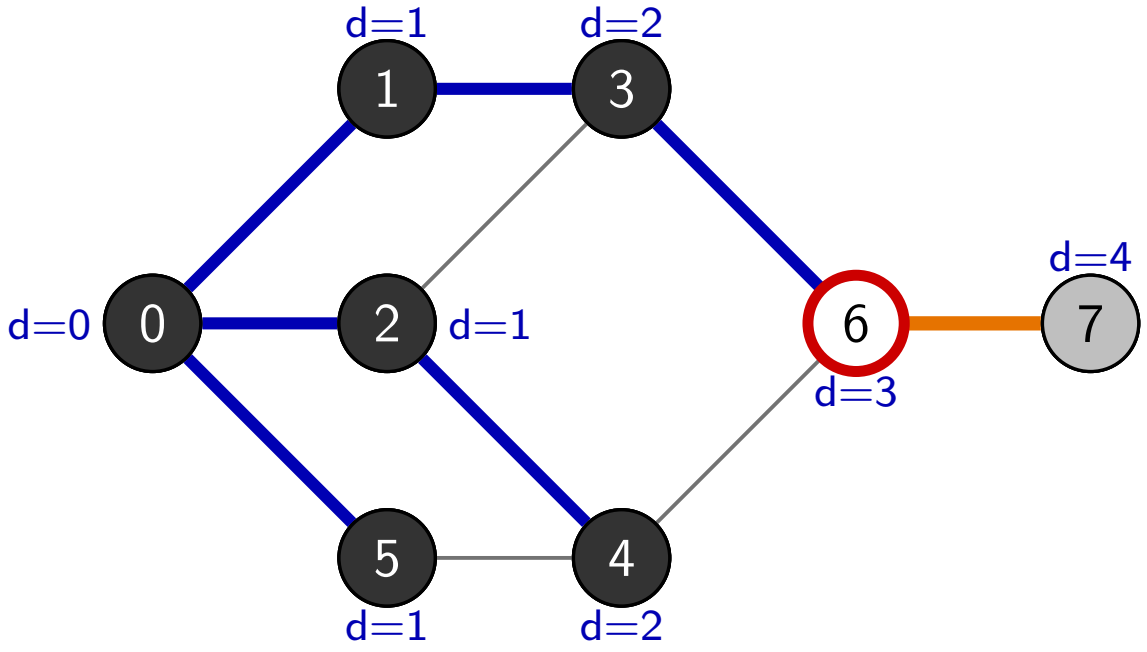
v	dist	pai
0	0	-1
1	1	0
2	1	0
3	2	1
4	2	2
5	1	0
6	3	3
7		

fila: 6

retira 4, lista [2 5 6]: todos marcados, nada entra

na fila já retirado retirado agora

Busca em largura passo a passo



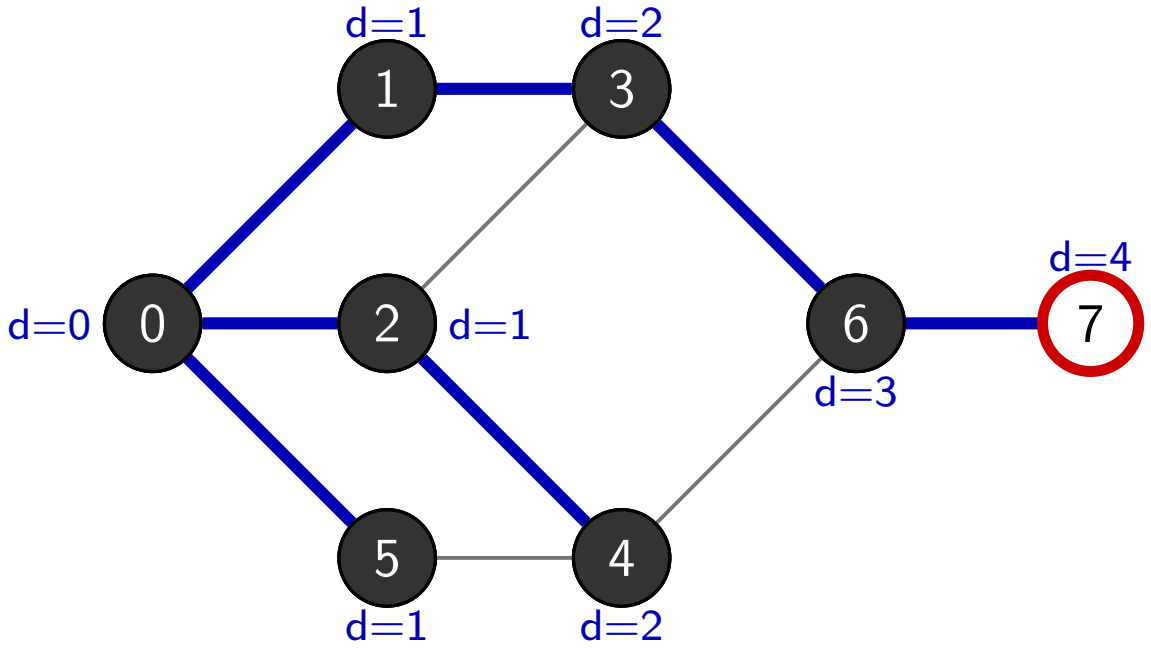
v	dist	pai
0	0	-1
1	1	0
2	1	0
3	2	1
4	2	2
5	1	0
6	3	3
7	4	6

fila: 7

retira 6, lista [3 4 7]: 7 entra na fila

na fila já retirado retirado agora

Busca em largura passo a passo



v	dist	pai
0	0	-1
1	1	0
2	1	0
3	2	1
4	2	2
5	1	0
6	3	3
7	4	6

fila:

retira 7, lista [6]: fila vazia, fim da busca

na fila já retirado retirado agora

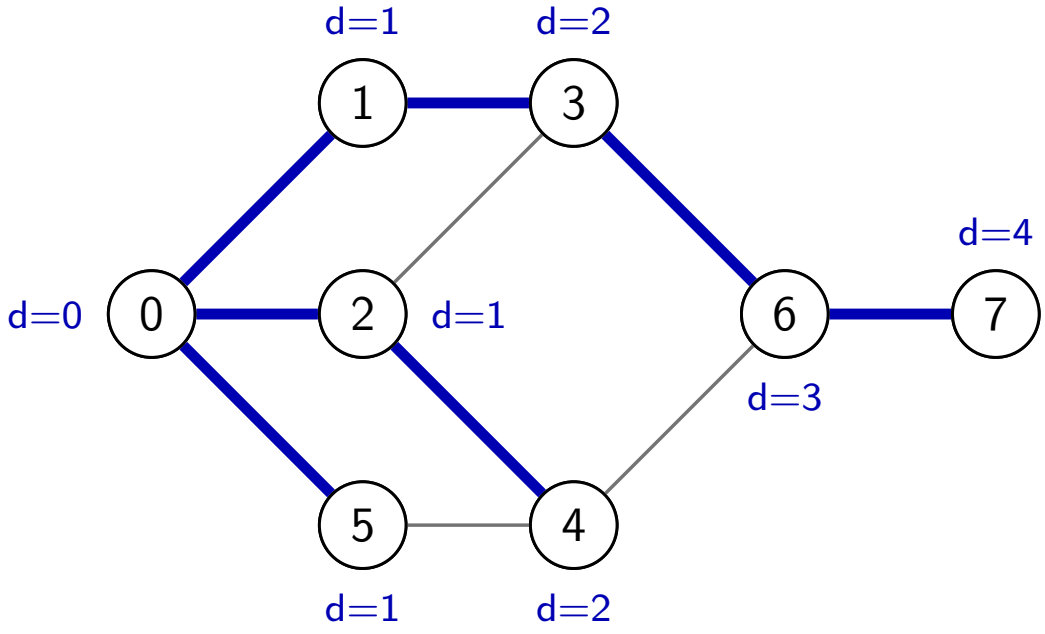
Busca em largura em G, a partir de 0

```
retira 0   fila: [1 2 5]
retira 1   fila: [2 5 3]
retira 2   fila: [5 3 4]
retira 5   fila: [3 4]
retira 3   fila: [4 6]
retira 4   fila: [6]
retira 6   fila: [7]
retira 7   fila: []
```

```
 v | dist | pai
---+------+----
0  |    0 |  -1
1  |    1 |   0
2  |    1 |   0
3  |    2 |   1
4  |    2 |   2
5  |    1 |   0
6  |    3 |   3
7  |    4 |   6
```

- ao retirar o 2: o 0 e o 3 já estão marcados, só o 4 entra;
- ao retirar o 5: nada entra;
- é o percurso em largura do material de árvores, com o vetor `marcado`: em grafo, um vértice pode ser alcançado por mais de um caminho.

Árvore da busca em largura



As arestas `pai[w]-w` formam uma árvore. Os vértices saem da fila em ordem de distância: 0; depois 1, 2, 5; depois 3, 4; depois 6; depois 7.

Custo: cada vértice entra uma vez na fila, e cada lista é percorrida uma vez. $O(V + E)$ com listas, $O(V^2)$ com matriz.

Caminho com menos arestas

- um vértice à distância $d + 1$ só entra na fila quando um vizinho à distância d sai, e todos os mais próximos já entraram;
- `dist[v]` é o menor número de arestas de s até v.

```
func (b *BFS) Caminho(v int) []int {
  if !b.marcado[v] {
    return nil
  }
  var cam []int
  for x := v; x != -1; x = b.pai[x] {
    cam = append(cam, x)
  }
  slices.Reverse(cam)
  return cam
}
```

- de 0 até 7: `[0 1 3 6 7]`, 4 arestas. O caminho 0, 2, 3, 6, 7 também tem 4: a busca garante o número de arestas, e não o caminho.

Aplicações da busca em largura

- **graus de separação**: conexões de 1º, 2º e 3º grau do LinkedIn; número de Kevin Bacon (atores ligados por um filme em comum); número de Erdős (coautoria de artigos);
- **redes de computadores**: rota com menos saltos (*hops*) entre dois roteadores;
- **coletor da web** (*web crawler*): fila de páginas e conjunto de endereços já vistos; os vizinhos de uma página só aparecem depois que ela é baixada;
- **quebra-cabeça de 8 peças**: cada arrumação do tabuleiro é um vértice, cada movimento é uma aresta; 181.440 arrumações, nenhuma a mais de 31 movimentos da solução.

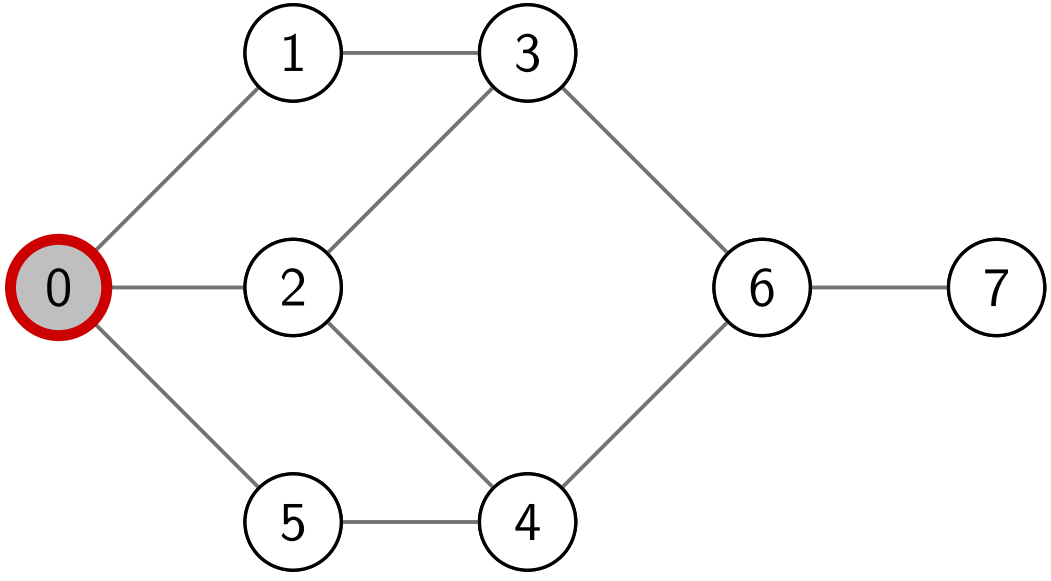
Em todos os casos, cada passo tem o mesmo custo. Passos com custos diferentes são o assunto da aula de caminhos mínimos.

Busca em profundidade (DFS)

```
func (d *DFS) visita(v int) {
  d.marcado[v] = true
  for _, w := range d.g.adj[v] {
    if !d.marcado[w] {
      d.pai[w] = v
      d.visita(w)
    }
  }
}
```

- vai o mais longe possível por um caminho, e volta quando não há vizinho novo;
- a pilha de chamadas faz o papel da fila da busca em largura;
- custo: $O(V + E)$ com listas, pelo mesmo argumento.

Busca em profundidade passo a passo



`visita(0)`

pilha de chamadas

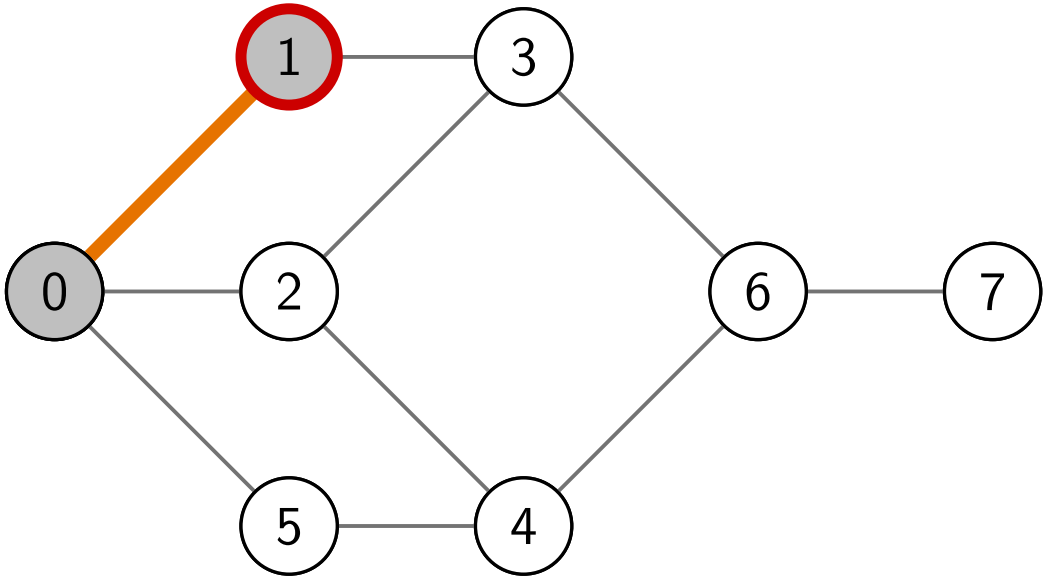
pré-ordem: 0

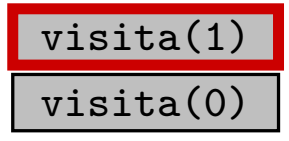
pós-ordem:

`visita(0)`: lista `[1 2 5]`, desce para o 1

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

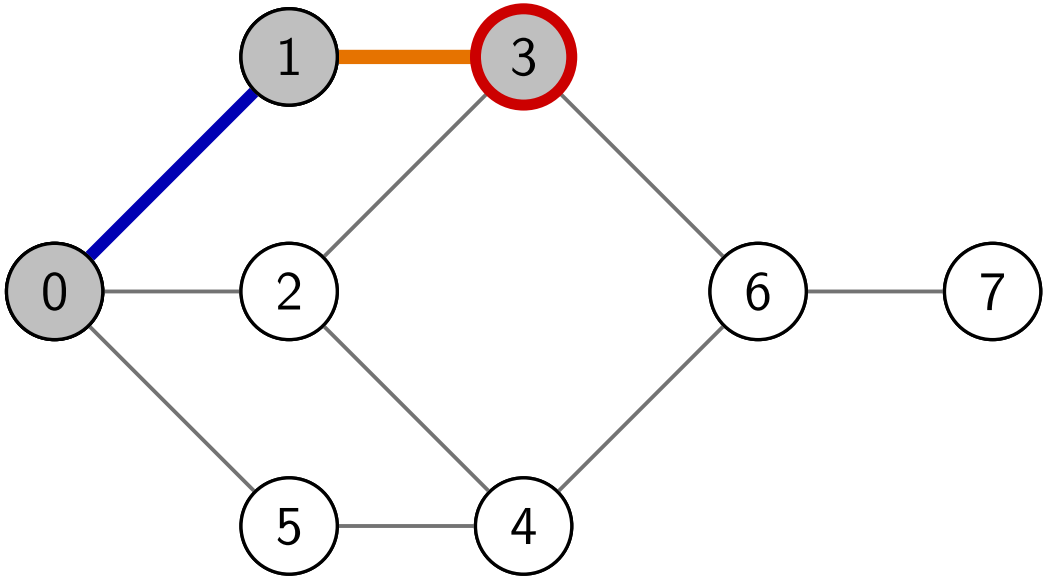
pré-ordem: `0 1`

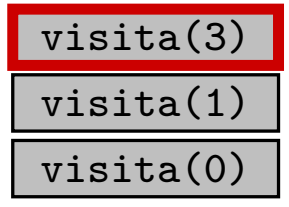
pós-ordem:

`visita(1)`: lista `[0 3]`, 0 já marcado, desce para o 3

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

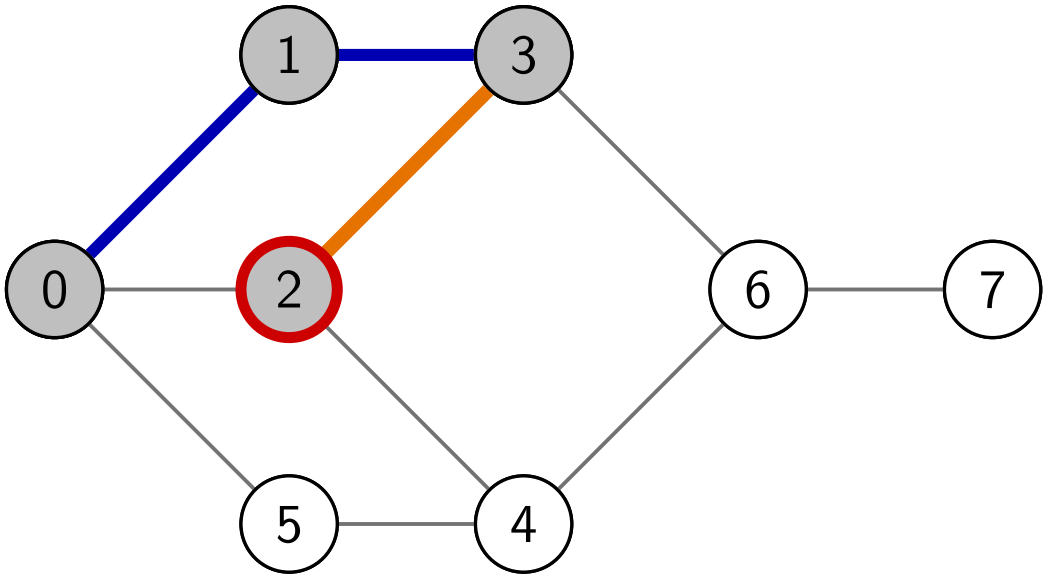
pré-ordem: `0 1 3`

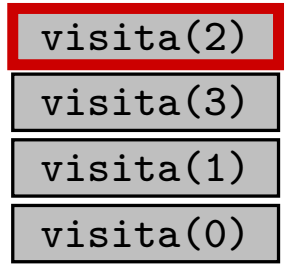
pós-ordem:

`visita(3)`: lista `[1 2 6]`, 1 já marcado, desce para o 2

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

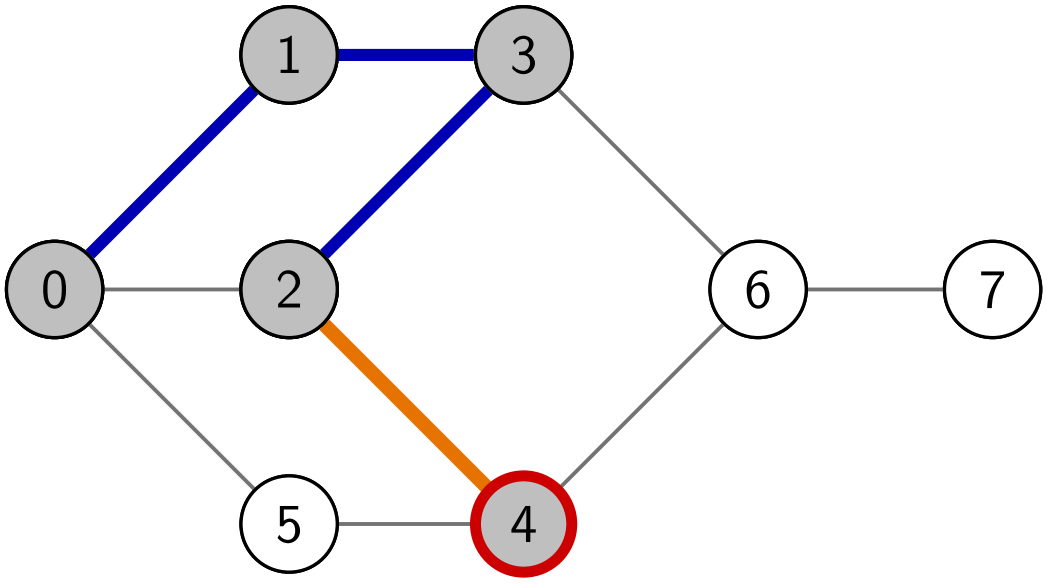
pré-ordem: `0 1 3 2`

pós-ordem:

`visita(2)`: lista `[0 3 4]`, 0 e 3 já marcados, desce para o 4

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

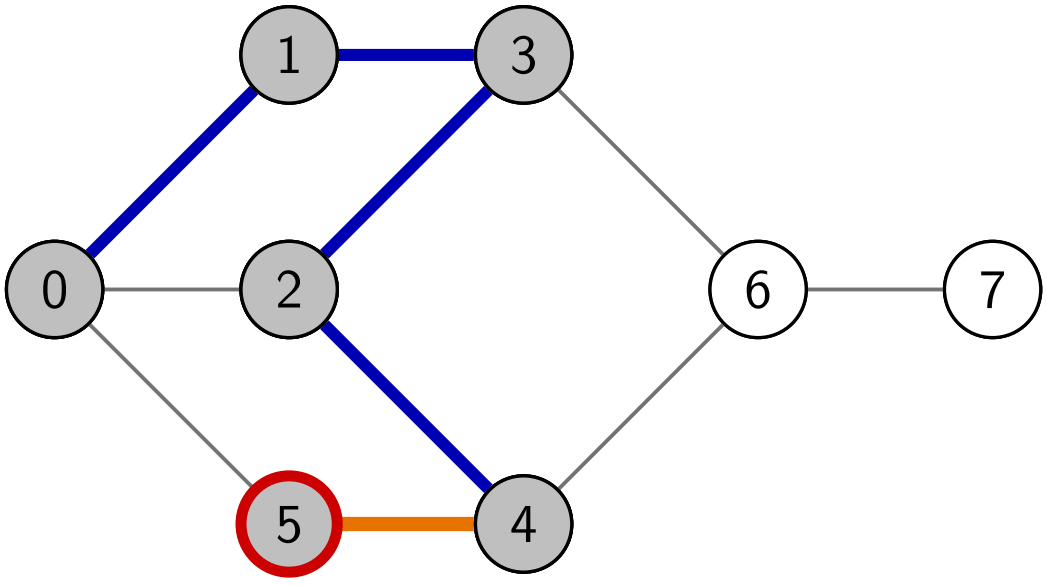
pré-ordem: `0 1 3 2 4`

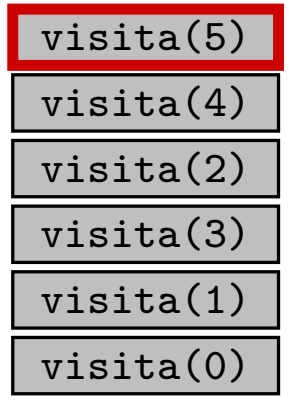
pós-ordem:

`visita(4)`: lista `[2 5 6]`, 2 já marcado, desce para o 5

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

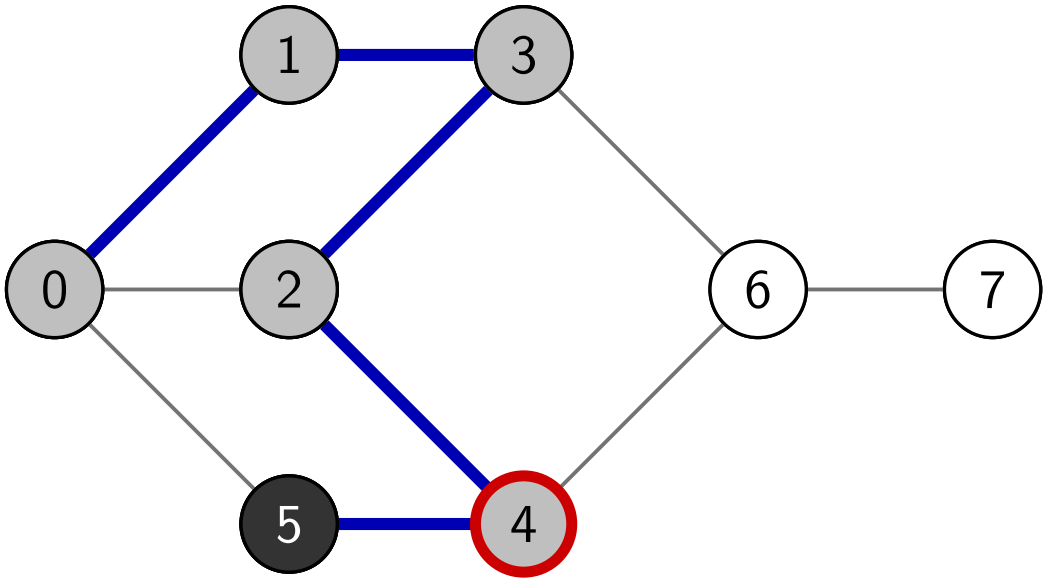
pré-ordem: `0 1 3 2 4 5`

pós-ordem:

`visita(5)`: lista `[0 4]`, todos marcados

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

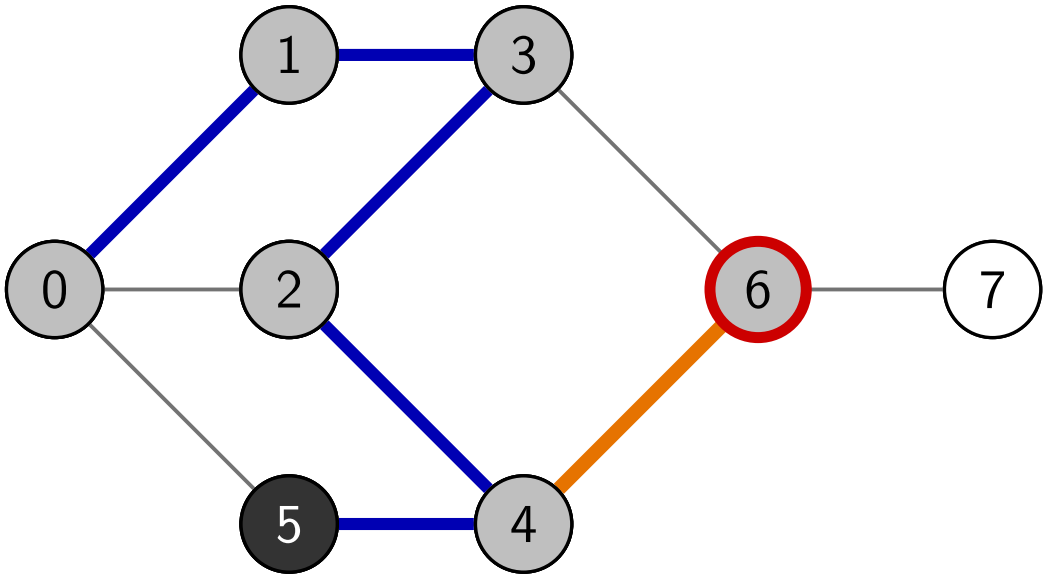
pré-ordem: 0 1 3 2 4 5

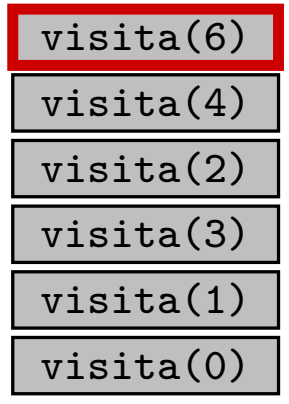
pós-ordem: 5

`termina` 5: volta ao 4; o 6 não está marcado

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

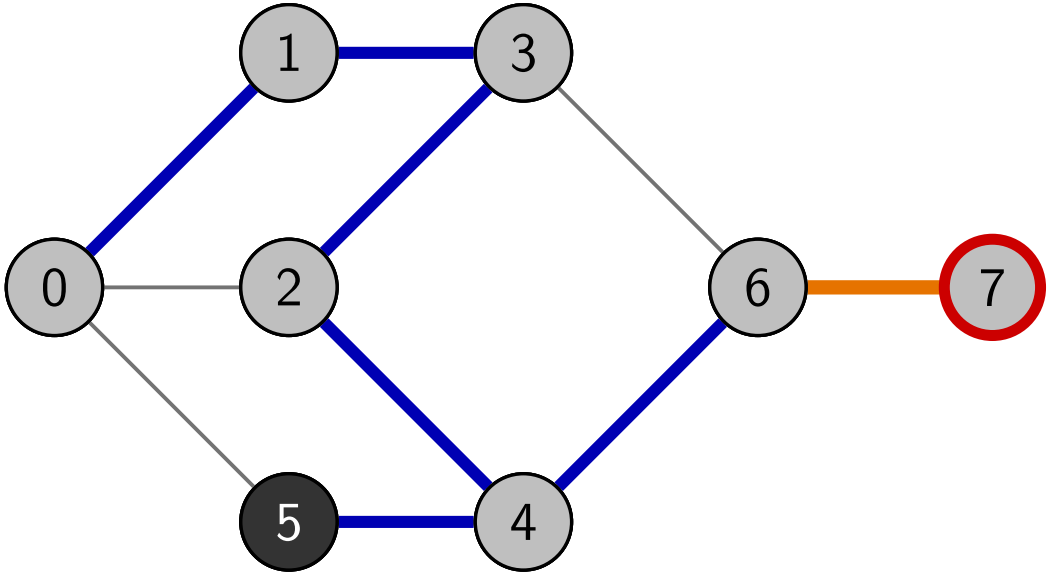
pré-ordem: `0 1 3 2 4 5 6`

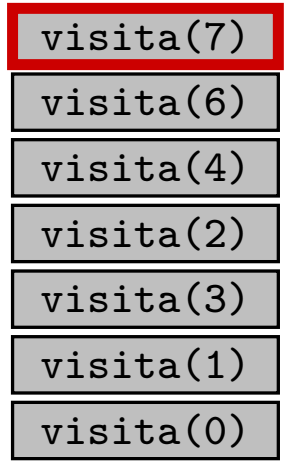
pós-ordem: `5`

`visita(6)`: lista `[3 4 7]`, 3 e 4 já marcados, desce para o 7

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

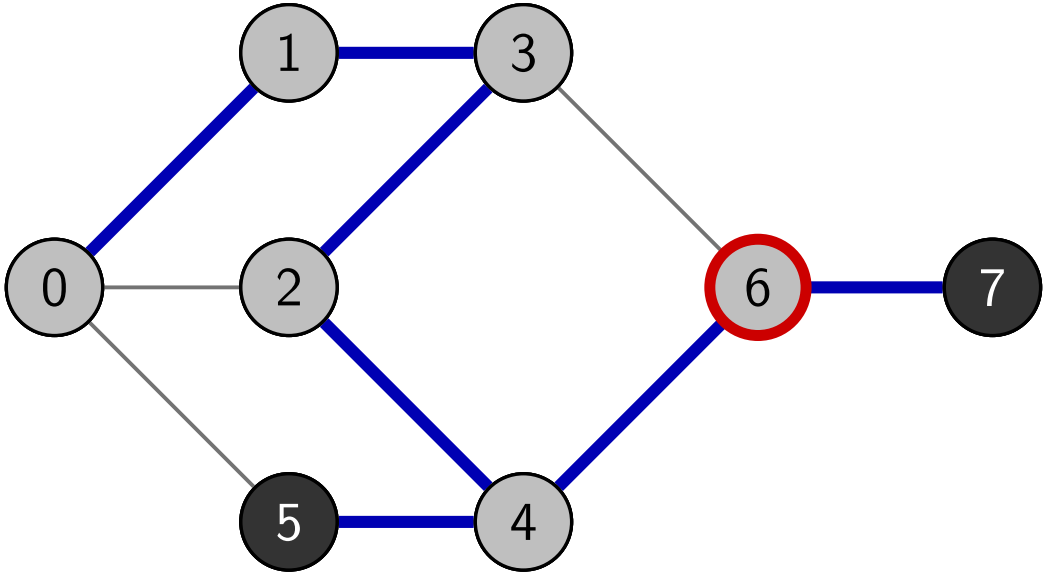
pré-ordem: `0 1 3 2 4 5 6 7`

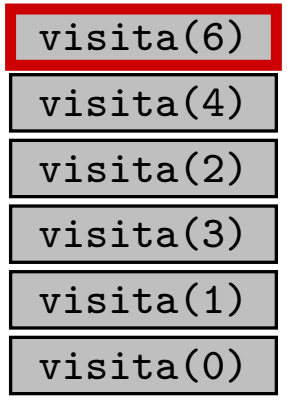
pós-ordem: `5`

`visita(7)`: lista `[6]`, todos marcados

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

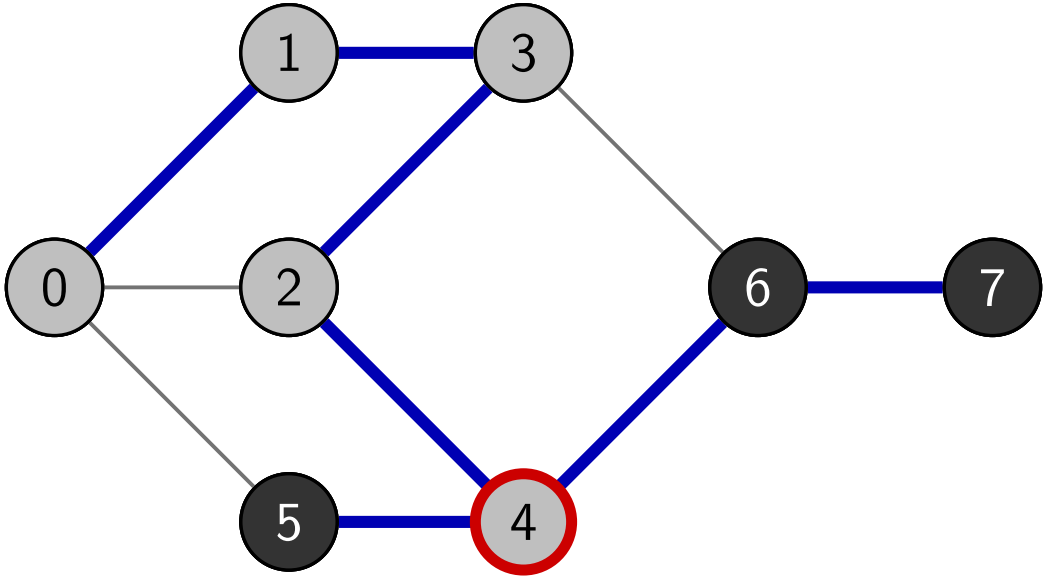
pré-ordem: 0 1 3 2 4 5 6 7

pós-ordem: 5 7

`termina` 7: volta ao 6; fim da lista

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

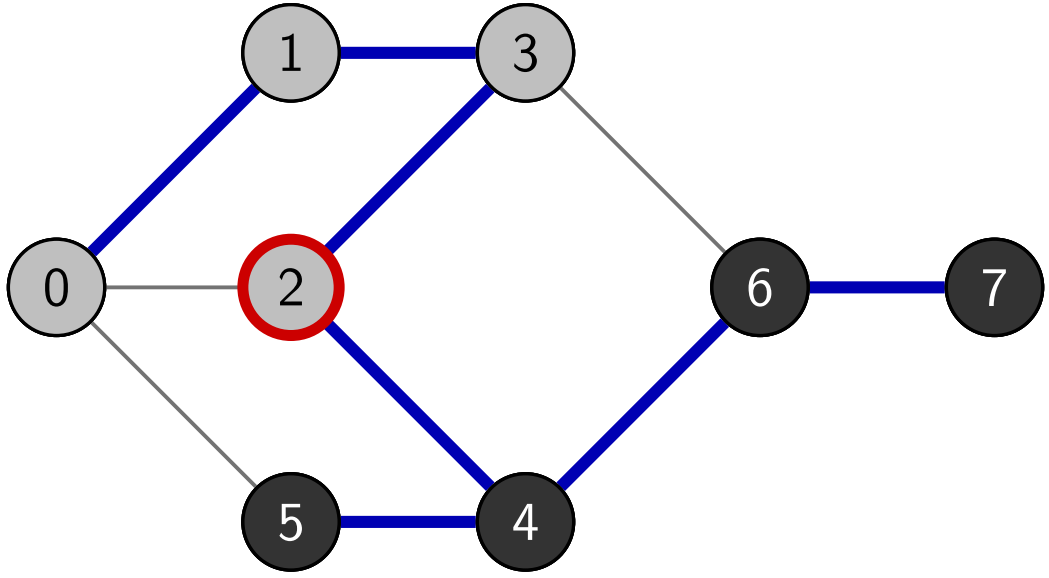
pré-ordem: 0 1 3 2 4 5 6 7

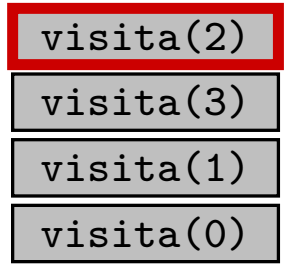
pós-ordem: 5 7 6

`termina 6`: volta ao 4; fim da lista

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

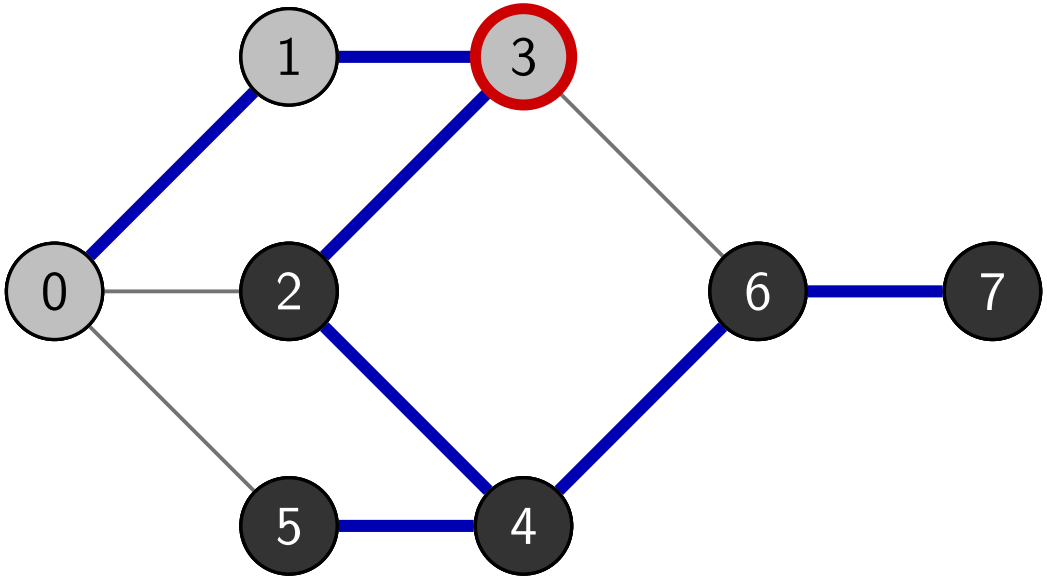
pré-ordem: 0 1 3 2 4 5 6 7

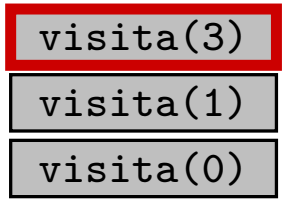
pós-ordem: 5 7 6 4

`termina` 4: volta ao 2; fim da lista

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

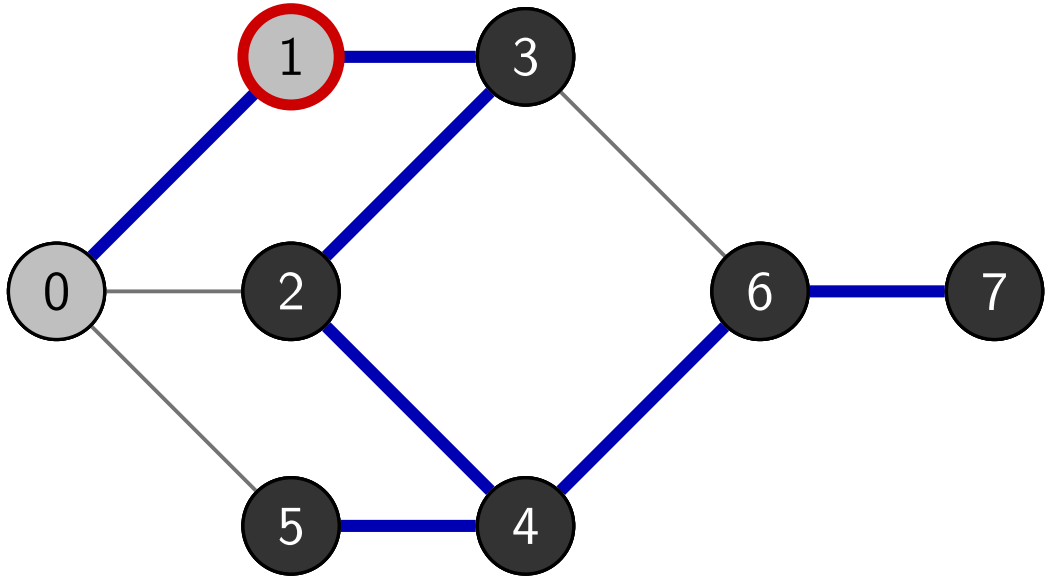
pré-ordem: 0 1 3 2 4 5 6 7

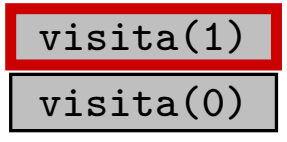
pós-ordem: 5 7 6 4 2

`termina` 2: volta ao 3; 6 já marcado, fim da lista

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo





pilha de chamadas

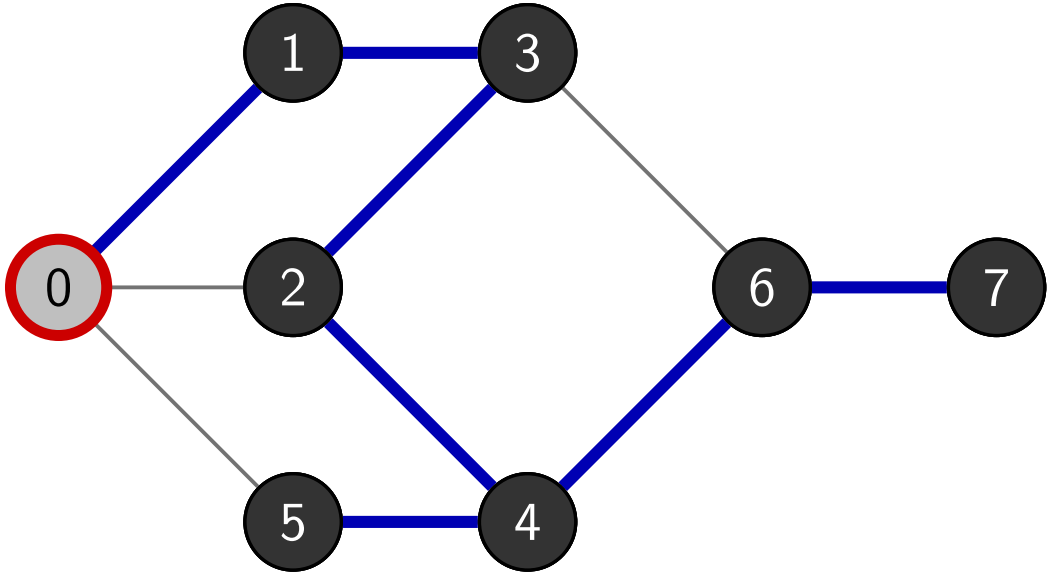
pré-ordem: 0 1 3 2 4 5 6 7

pós-ordem: 5 7 6 4 2 3

`termina` 3: volta ao 1; fim da lista

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo



`visita(0)`

pilha de chamadas

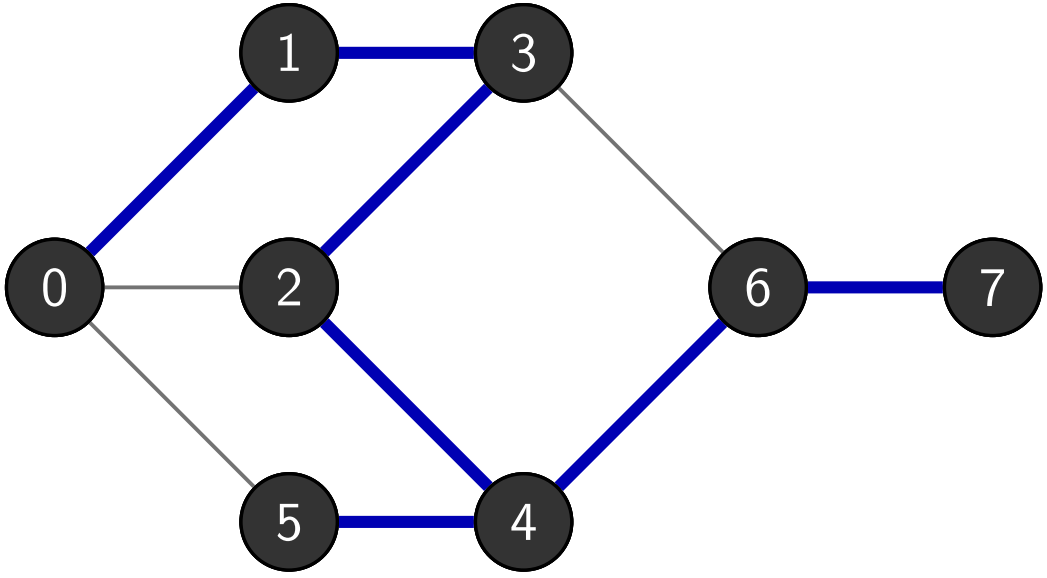
pré-ordem: `0 1 3 2 4 5 6 7`

pós-ordem: `5 7 6 4 2 3 1`

`termina 1`: volta ao 0; 2 e 5 já marcados, fim da lista

chamada em andamento chamada terminada em execução

Busca em profundidade passo a passo



pilha de chamadas

pré-ordem: 0 1 3 2 4 5 6 7

pós-ordem: 5 7 6 4 2 3 1 0

`termina 0`: fim da busca

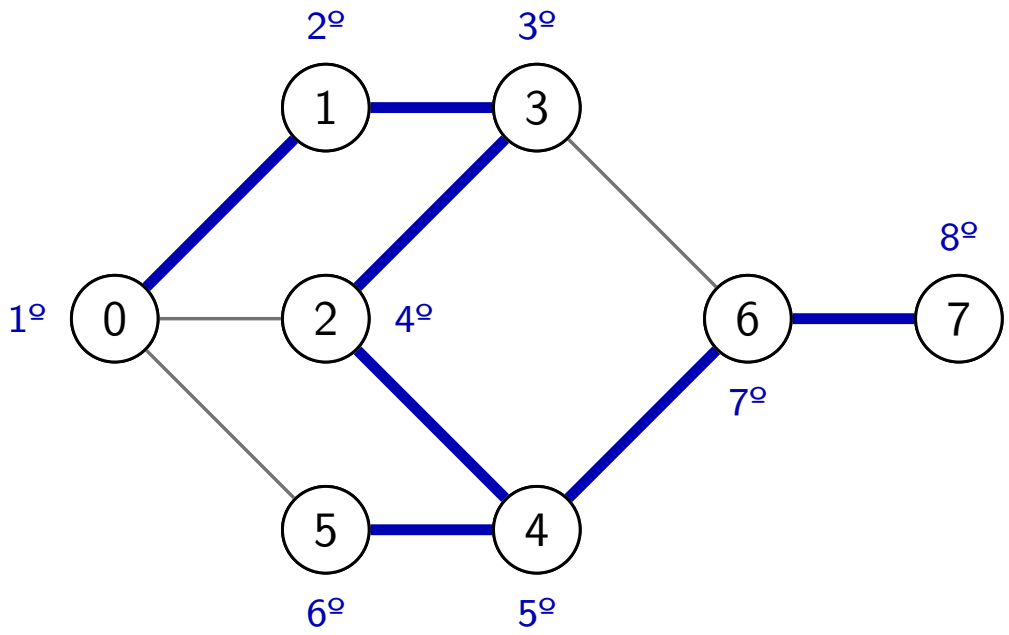
chamada em andamento chamada terminada em execução

Busca em profundidade em G, a partir de 0

```
visita(0)
  visita(1)
    visita(3)
      visita(2)
        visita(4)
          visita(5)
          termina 5
          visita(6)
            visita(7)
            termina 7
          termina 6
        termina 4
      termina 2
    termina 3
  termina 1
termina 0
```

- de 3, lista `[1 2 6]`: desce para o 2, e não para o 6;
- o 6 só é alcançado pelo 4;
- **pré-ordem**, quando a chamada começa:
  0 1 3 2 4 5 6 7
- **pós-ordem**, quando a chamada termina:
  5 7 6 4 2 3 1 0
- na pós-ordem, o 4 vem depois de 5, 6 e 7, que ele alcança.

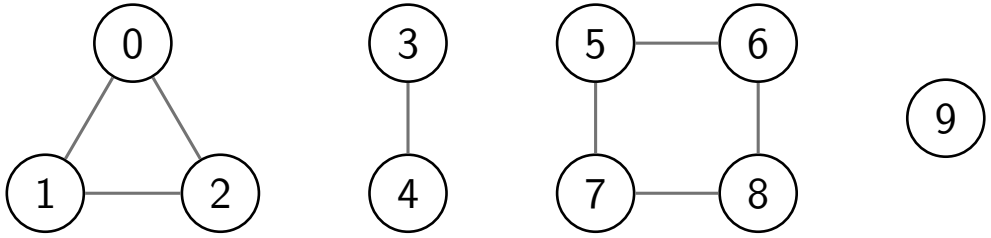
Árvore da busca em profundidade



Caminho de 0 até 7 pelo vetor `pai`: 0, 1, 3, 2, 4, 6, 7, com 6 arestas. A busca em largura achou um com 4. A busca em profundidade não dá garantia nenhuma sobre o número de arestas.

Componentes conexas

```
quantas := 0
for v := 0; v < g.V(); v++ {
  if !marcado[v] {
    visita(v, quantas) // marca tudo o que v alcança, com id = quantas
    quantas++
  }
}
```



id 0 id 1 id 2 id 3

v	0	1	2	3	4	5	6	7	8	9
id	0	0	0	1	1	2	2	2	2	3

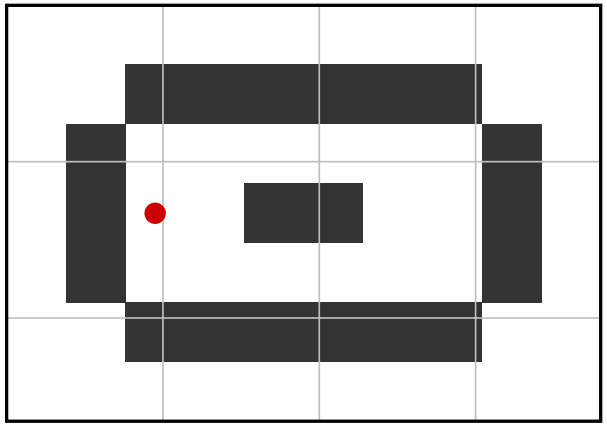
Componentes conexas e Union-Find

Mesmo papel do `id` do Quick-Find: v e w estão conectados se, e somente se, `id[v] == id[w]`.

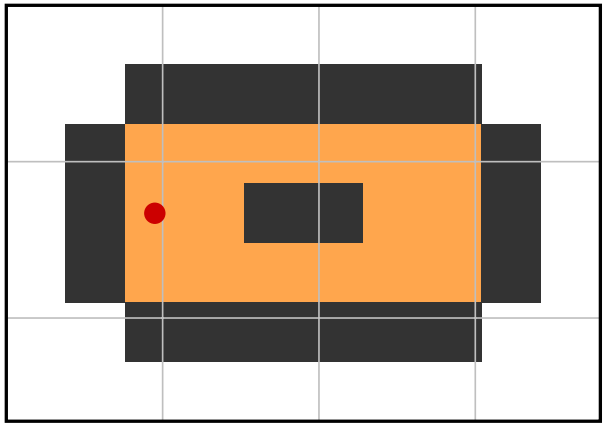
Busca	Union-Find
precisa do grafo inteiro guardado	recebe as arestas uma de cada vez, sem guardá-las
$O(V + E)$ uma vez, depois $O(1)$ por consulta	consultas entre uma aresta e outra

O algoritmo de Kruskal, na próxima aula, usa o Union-Find desse segundo jeito.

Componentes em imagens



antes do clique



depois do clique

- cada pixel é um vértice, ligado aos vizinhos de mesma cor: grafo implícito em grade, como o do labirinto;
- **balde de tinta** (*flood fill*): pinta a componente do pixel clicado; a varinha mágica seleciona da mesma forma;
- imagem de microscópio: cada componente de pixels claros é uma partícula, e contar componentes é contar partículas.

Detecção de ciclo

Na busca em profundidade, a aresta v-w examinada leva a três casos:

- w não marcado: desce para w;
- w marcado e pai de v: a aresta pela qual a busca chegou a v;
- w marcado e não é o pai: outro caminho de w até v. A aresta fecha um **ciclo**.

```
$ go run componentes.go
Ciclo: a aresta 2-0 leva a um vértice já marcado
Sem as arestas 1-2 e 7-8, tem ciclo? false
```

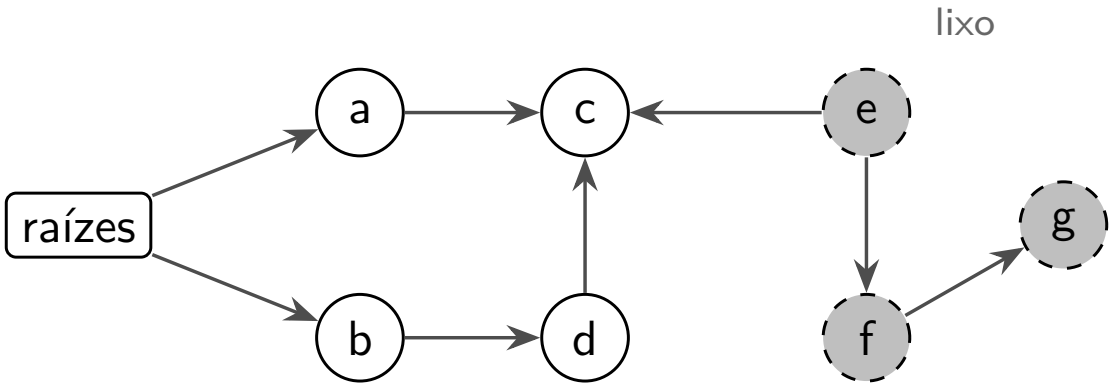
A busca desce 0, 1, 2. Na lista do 2, o 0 está marcado e o pai do 2 é o 1: ciclo 0, 1, 2.

Grafos dirigidos

```
func (g *Digrafo) AdicionaArco(v, w int) {
  g.adj[v] = append(g.adj[v], w)
}
```

- o arco $v \to w$ entra só na lista de v;
- as duas buscas funcionam sem mudança, seguindo o sentido dos arcos;
- a busca a partir de s marca os vértices **alcançáveis** a partir de s;
- w pode ser alcançável a partir de v sem que v seja alcançável a partir de w.

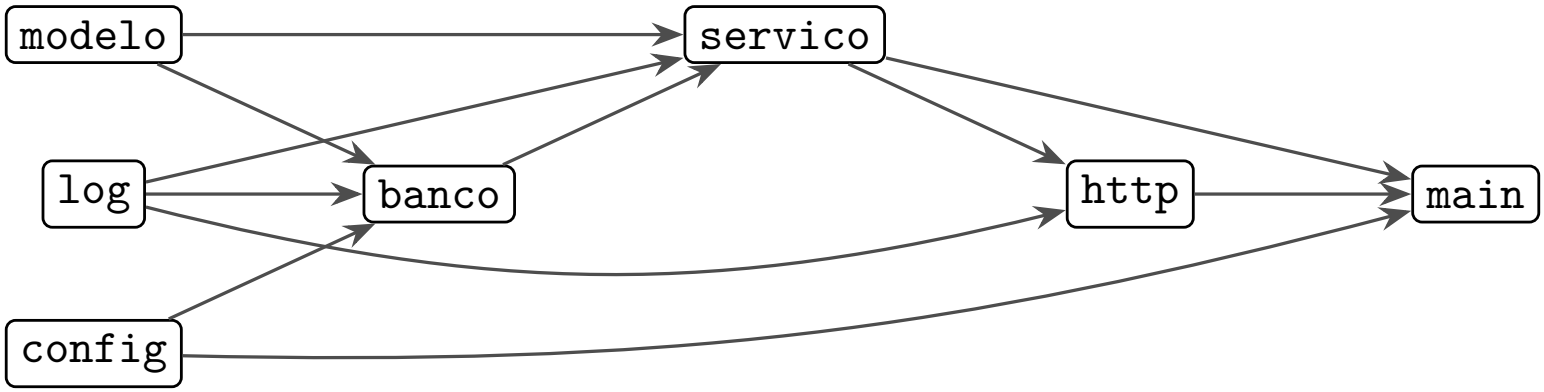
Alcançabilidade: coleta de lixo



- objetos são vértices, ponteiros são arcos; as **raízes** são as variáveis locais e globais;
- **marca e varre** (*mark-sweep*, McCarthy, 1960): marca o que as raízes alcançam e libera o resto;
- o coletor de lixo de Go é desse tipo; o código do `runtime` usa branco, cinza (ponteiros por examinar) e preto;
- análise de código: um trecho que o início da função não alcança nunca executa (`go vet: unreachable code`).

Dependências entre pacotes

Arco $B \to A$: o pacote A importa B, e B é compilado antes.



Ordenação topológica: uma ordem dos vértices em que todo arco aponta para a frente. Aqui, uma ordem de compilação.

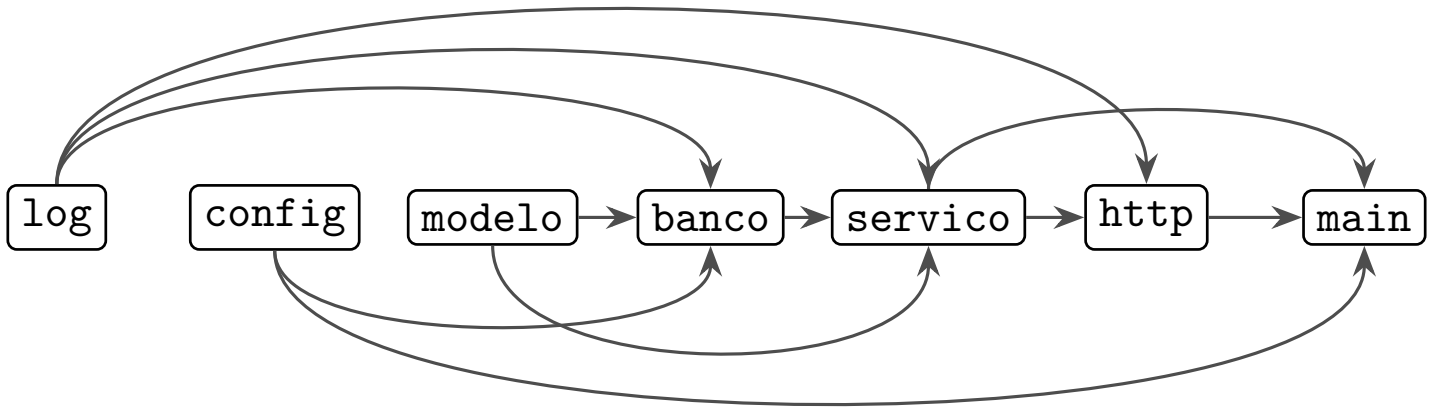
Pós-ordem invertida

- grafo sem ciclo: quando a chamada de v termina, tudo o que v alcança já terminou, dentro da chamada de v ou antes dela;
- na pós-ordem, cada vértice aparece depois de tudo o que ele alcança;
- invertida, cada vértice aparece antes de tudo o que ele alcança: é uma ordenação topológica.

Uma busca a partir de cada vértice ainda não visitado, como em componentes, e no fim a pós-ordem invertida.

Ordem de compilação

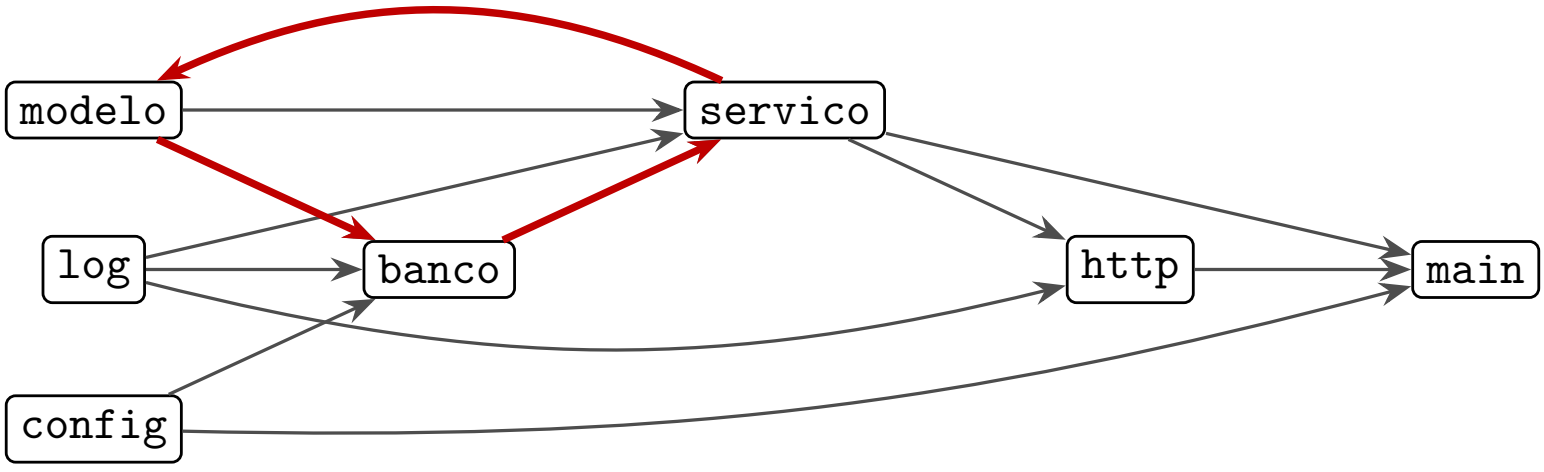
```
$ go run topologica.go
Ordem de compilação: [log config modelo banco servico http main]
```



Todos os arcos apontam para a direita. A ordem não é única: `config log modelo banco servico http main` também serve.

Ciclo em grafo dirigido

`modelo` passa a importar `servico`:



- nenhuma ordem resolve: ordenação topológica existe se, e somente se, não há ciclo (**DAG**, *directed acyclic graph*);
- a regra "marcado e não é o pai" falha: sem esse arco, a busca a partir de `modelo` já termina o `servico` via `banco`, e depois o arco `modelo -> servico` chega a um vértice marcado sem ciclo nenhum.

Três estados

- **branco**: ainda não visitado;
- **cinza**: chamada em andamento, ainda na pilha de chamadas;
- **preto**: chamada terminada.

Arco $v \to w$ com w cinza: v foi alcançado a partir de w, e o arco volta. Ciclo. Arco para vértice preto não fecha ciclo.

```
$ go run topologica.go
Com modelo importando servico:
  ciclo: [modelo banco servico]

$ go build .
...
  imports ciclo/b from a.go
  imports ciclo/a from b.go: import cycle not allowed
```

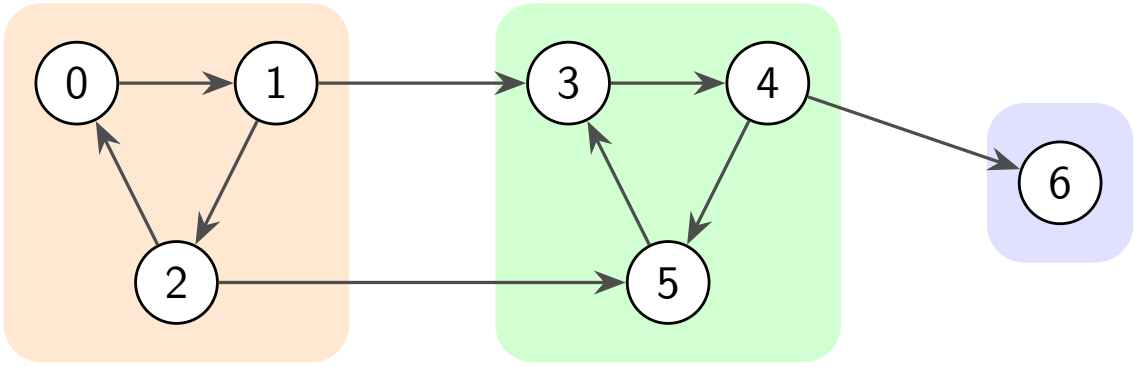
Ordenação topológica e ciclos em outras ferramentas

Onde	Vértice	Arco	Ciclo
`make`	alvo	pré-requisito	`Circular c <- a dependency dropped.`
planilha	célula	fórmula que usa a célula	`Err:522`, referência circular (LibreOffice)
grade curricular	disciplina	pré-requisito	nenhum aluno conclui o curso
compilador de Go	tipo	`struct` dentro de `struct`	`invalid recursive type`
compilador de Java	classe	herança	`cyclic inheritance`

Sem ciclo, uma ordenação topológica dá a ordem de construção, de recálculo ou de matrícula.

Componentes fortemente conexas

v e w **fortemente conectados**: cada um alcança o outro.



- do 3 não se volta ao 1; o 6 fica sozinho;
- ruas de mão única: em cada componente, vai-se de qualquer cruzamento a qualquer outro e volta-se;
- trocando cada componente por um vértice, o grafo não tem ciclo;
- Kosaraju-Sharir e Tarjan: $O(V + E)$, com busca em profundidade. Fora desta disciplina.

De volta ao labirinto: grafo implícito

```
func vizinhos(v int) []int {
  l, c := v/colunas, v%colunas
  var viz []int
  for _, d := range [][2]int{{0, 1}, {1, 0}, {0, -1}, {-1, 0}} {
    nl, nc := l+d[0], c+d[1]
    if mapa[nl][nc] != '#' {
      viz = append(viz, celula(nl, nc))
    }
  }
  return viz
}
```

- vértice: a célula `linha*colunas + coluna`;
- sem listas guardadas: os vizinhos são calculados quando a busca pede. As buscas trocam `g.adj[v]` por `vizinhos(v)`;
- a grade da etapa 2 do trabalho prático é um grafo desse tipo.

De volta ao labirinto: os dois caminhos

- busca em largura: 13 passos, porque `dist[E]` é 13, o menor número de arestas de `S` até `E`;
- busca em profundidade: vizinhos na ordem direita, abaixo, esquerda, acima. Segue pela região aberta da direita antes de voltar para `E`, e o vetor `pai` registra 45 passos;
- células visitadas: 43 e 48, de 92 livres. Nenhuma das duas garante visitar menos; depende da posição de `E` e da ordem dos vizinhos.

Escolher entre as buscas

Problema	Busca
caminho com menos arestas	em largura
vértices alcançáveis	qualquer uma
componentes conexas	qualquer uma
ciclo, grafo não dirigido	qualquer uma
ciclo, grafo dirigido	em profundidade, três estados
ordenação topológica	em profundidade, pós-ordem invertida

Para a próxima aula

- exercícios 1 a 11 do roteiro da aula, sem entrega e sem nota. O conteúdo cai na Prova 2;
- rodar `representacao.go`, `bfs.go`, `dfs.go`, `componentes.go` e `topologica.go`, trocando as arestas;
- leitura: Sedgewick, seções 4.1 e 4.2; Cormen, capítulo 22.

Próxima aula: árvores geradoras mínimas, com os algoritmos de Kruskal e de Prim. As arestas passam a ter peso.